Campus Supply Desk - A2

Mailles Machiana / COMP3415 / November 8, 2026

Repo: https://github.com/McMerger/COMP3415-A2-Campus-Supply-Desk

What the program does

Campus Supply Desk is a small terminal program for ordering campus merchandise. You can look at stock, get a quote, place an order and cancel it later. Orders and stock are saved in JSON. Money is stored in cents, so $12 is 1200 in the code.

__main__.py starts cli.py. With no command, it opens menu.py. Both call OrderService in service.py. That's where the prices and stock changes happen. storage.py reads and writes the save, and catalog.json supplies the starting products when there isn't a save yet.

A quote doesn't reserve stock. An order has to check again before it saves. Cancelling should put the items back once and leave the old order in history. The functional spec was used to check these rules, rather than assuming the starter's output was right.

Problems that needed attention

These references are to supplydesk/service.py at the starting commit 82ed75e. They come from the work after Checkpoint A.

The same calculations were written twice

Quote and place repeat validation and pricing in lines 15-60 and 65-112. A change could easily be made in one copy and missed in the other. Both now use _build_quote.

Exactly $50 missed the student discount

Lines 48 and 98 say > 5000. Two $25 bags should get a $5 discount for a student, but didn't. Using >= fixes that edge case.

Duplicate items could use more stock than was available

Lines 36 and 86 only check one line at a time. With 20 notebooks, lines of 11 and 10 both pass. The fix adds up each product's requested quantity before allowing an order.

Cancelling twice returned the same items twice

Lines 142-147 add stock back every time. Two mugs cancelled twice leave 10 in stock instead of 8. An already-cancelled order now returns without changing stock.

More repeated loops and numbers

The order searches in lines 128-141 and stock loops in 114-117 and 142-145 were pulled into _find_order and _change_stock. Values such as 5000, 10000 and 600 also got names, so the discount and delivery rules are easier to find. These were two separate maintenance problems: repeated operations and repeated pricing numbers.

What changed, and how it was checked

The repeated code was moved first. At that point, the same six failing assertions were still there. The three bugs were fixed after that, and all 27 tests at that stage passed. This kept the refactor separate from the changes in behavior.

The tests use temporary saves. Expected prices come from the spec, not from the pricing code itself. Otherwise, a test could just repeat the same mistake.

The runs went like this:

- Starter: 4 tests passed.
- Added baseline tests: 27 tests, with 6 failures.
- Shared-code refactor: the same 6 failures.
- Bug fixes: all 27 passed.
- Express tests before support: 49 tests, with 5 failures and 8 errors.
- Completed version: all 49 passed, including the original four.

Logs 01–07 in `evidence/` keep those results. Some tests use subtests, so one method can check several inputs.

The important cases were the $50 discount, free standard delivery at $100 after discount, bad quantities, duplicate products and cancelling twice. Other checks make sure a rejected order doesn't change the save or use an ID. Reopening a save must keep stock, history and the prices originally charged.

Command tests run real Python processes. Menu tests use scripted input to check defaults, errors and confirmation. Seven wrong changes were also tried in separate copies, and the tests caught all seven. Their output is in `evidence/08-targeted-mutations.json`.

CR-01: express delivery

Express is available in the service, commands and menu. It always adds $12. There's no free-express threshold, and the student discount doesn't apply to the fee. Regular customer and pickup are still the defaults. The menu prompt, command help and README now mention express.

The client examples give these totals, and the tests match them:

- Two notebooks, regular, express: $10 + $12 = $22.
- Two hoodies, student, express: $80 − $8 + $12 = $84.
- Three hoodies, regular: $120 with standard delivery, $132 with express.

The tests also reopen and cancel express orders. Original items and totals stay recorded, stock comes back once, and order IDs aren't reused. Other fulfillment values are still rejected.

AI suggestions - which ones made sense?

The separate AI note records the assistance and options considered. The useful part wasn't just getting suggestions, but checking whether they fitted this program and its rules.

Sharing the quote calculations made sense. There was real duplication to remove, and the helper could use the state already loaded by the caller. Tests compare quotes and orders to check that they still agree.

Using a dictionary for stock updates was also useful, but using one directly on raw product IDs was less straightforward. A list or dictionary used as an ID can cause TypeError. The service should raise ValueError for bad requests instead. With five products, keeping the small validation scan was fine.

`round(subtotal * 0.1)` looked like a quick way to calculate the discount. The problem is that Python rounds ties to even, while the spec says half cents go up. The integer calculation stayed. A test price of 5005 cents needs a 501-cent discount. The normal catalog can't reach that half-cent case, so it needs a fixture.

Express couldn't just reuse the standard-delivery threshold with a higher fee. That would make some express orders free. Keeping a separate branch was simpler and followed CR-01. Tests check that the fee is never waived or discounted.

A database or a set of delivery classes seemed unnecessary for this change. JSON and a short conditional were enough. Duplicate lines also stay separate in results; their quantities are only combined for the stock check.

Where the work ended up

All 49 tests pass on Windows with Python 3.12.10. That doesn't cover every possible case. No coverage percentage was measured. Menu input, permission errors, EOF and Ctrl+C are simulated, and Python 3.10 wasn't tested separately. Concurrent writers and recovery from a crash while saving are outside the supplied spec.

To run the checks from the project folder:

```
python -m unittest discover -s tests -v
python scripts/verify.py
```

The repo keeps the code and commit history. Sources were the starter README, functional spec, CR-01 and saved test output. The full original handout wasn't available, so its exact formatting requirements still need checking.